

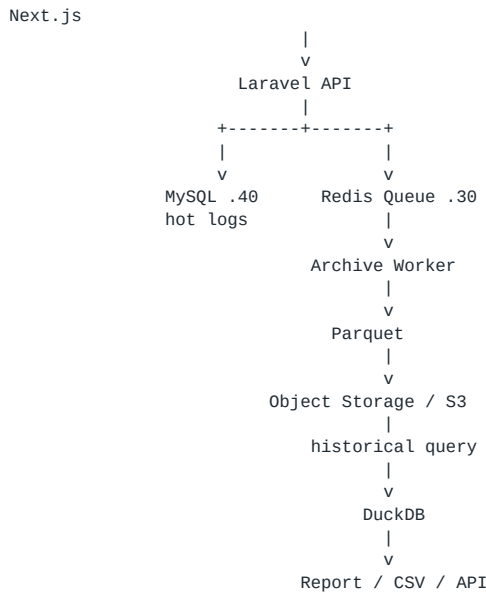
DuckDB for Activity Logs & Long-Term Analytics

Complete production guide for Laravel + MySQL + Redis + Parquet + Object Storage + DuckDB

Hot	MySQL — recent operational activity
Queue	Redis Queue — asynchronous archive/report jobs
Archive	Parquet — compressed columnar event files
Storage	S3-compatible object storage / MinIO
Analytics	DuckDB — embedded analytical SQL engine
Retention	5, 10, 20+ years without keeping all history in MySQL

Core idea: DuckDB is not a replacement for MySQL. It is an analytical engine that can query Parquet directly, including files in object storage. This makes it a strong low-operations solution for long-term activity history.

1. Architecture at a Glance



Responsibilities

- MySQL: recent activity queries, application-facing pagination, transactions.
- Redis Queue: asynchronous archive and report jobs.
- Parquet: durable compressed columnar representation.
- Object Storage: long-term retention and independent durable copy.
- DuckDB: SQL analytics over archived files.

Future evolution: add Kafka when event throughput/multiple consumers require streaming, and add ClickHouse when historical analytics needs high concurrency.

2. What DuckDB Is

DuckDB is an embedded analytical database/SQL engine. Unlike MySQL or ClickHouse, it does not need a separate database server for normal use. A worker or application process can open DuckDB and query local or remote analytical data.

A useful mental model is SQLite for analytics: optimized for scans, GROUP BY, joins, aggregations and data processing rather than transactional web workloads.

Workload	MySQL	DuckDB
Transactions	Excellent	Not the goal
Single-row lookups	Excellent	Not the goal
Large aggregations	Possible	Excellent
GROUP BY/time analytics	Good	Excellent
Query Parquet directly	Not native workflow	Excellent
High concurrent web traffic	Excellent	Limited vs server DB

3. Installation

For your architecture, a small Python analytics worker is the most practical starting point because Python has mature DuckDB integration and works naturally with Parquet.

```
python3 -m venv /opt/activity-analytics/.venv
source /opt/activity-analytics/.venv/bin/activate
pip install --upgrade pip
pip install duckdb pandas pyarrow
```

```
python -c "import duckdb; print(duckdb.__version__)"
```

For the CLI, install the current DuckDB release using the official DuckDB installation/download instructions for your CPU/OS, then verify:

```
duckdb --version
```

- A practical analytics host can start around 4–8 CPU cores and 16–32 GB RAM; benchmark before scaling.
- Use fast local NVMe for temporary/spill work.
- No public database port is required.

4. DuckDB SQL Basics

```
duckdb
SELECT version();
SELECT 1 + 2;
SELECT current_timestamp;
```

Query Parquet without importing it

```
SELECT COUNT(*)
FROM read_parquet('activity_logs/2026/*.parquet');
```

Multiple files

```
SELECT *
FROM read_parquet('activity_logs/**/*.parquet')
LIMIT 100;
```

Inspect schema

```
DESCRIBE
SELECT *
FROM read_parquet('activity_logs/2026/*.parquet');
```

5. Parquet Archive Design

Parquet is a columnar format designed for analytical storage. It is well suited to long-term activity data because it supports compression and efficient column reads.

```
activity_logs/
  year=2024/
    month=01/
      part-000001.parquet
      part-000002.parquet
    year=2025/
      month=01/
        ...
      year=2026/
        month=08/
          ...
```

- Partition primarily by event date.
- Avoid one file per event.
- Avoid millions of tiny files.
- Use reasonably sized files and compact small files periodically.
- Use ZSTD as a strong starting compression choice and benchmark.

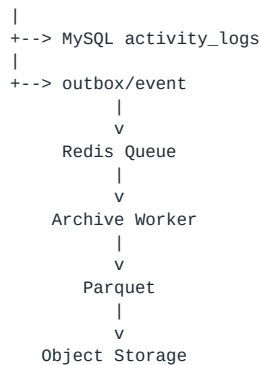
A useful starting target is tens to hundreds of MB per Parquet file. Exact sizing must be benchmarked for your event width, object store and query pattern.

Create Parquet with DuckDB

```
COPY (
  SELECT *
  FROM read_json_auto('raw/activity/*.json')
)
TO 'archive/activity_2026_08.parquet'
(FORMAT PARQUET, COMPRESSION ZSTD);
```

6. Archive Pipeline

```
Laravel
```



- Read bounded batches.
- Normalize types and schema.
- Create compressed Parquet.
- Upload only after the file is complete.
- Record manifest, row count, min/max timestamps and checksum.
- Only then mark the source batch archived.

Never publish a partially written archive file as complete.

7. Python Archive Worker Example

```

import duckdb
import pandas as pd
from pathlib import Path

def write_parquet(rows, output_file):
    df = pd.DataFrame(rows)
    con = duckdb.connect()
    con.register("events_df", df)
    con.execute(f'''
        COPY (SELECT * FROM events_df)
        TO '{output_file}'
        (FORMAT PARQUET, COMPRESSION ZSTD)
    ''')
    con.close()

rows = [{
    "event_id": "01J...",
    "tenant_id": 10,
    "user_id": 100,
    "action": "contact.updated",
    "subject_id": 200,
    "created_at": "2026-08-19 15:20:00"
}]

Path("/tmp/archive").mkdir(parents=True, exist_ok=True)
write_parquet(rows, "/tmp/archive/part-000001.parquet")
  
```

For large batches, stream or batch data instead of materializing an unbounded Python list.

8. Object Storage / S3 / MinIO

Object storage is the long-term system of record. DuckDB is the query engine used when historical data is needed.

```

s3://activity-archive/
activity_logs/
  year=2024/month=01/part-000001.parquet
  year=2025/month=12/part-000001.parquet
  year=2026/month=08/part-000001.parquet
manifests/
  year=2026/month=08/manifest.json
  
```

DuckDB can access object storage through its HTTP/S3-related extensions. Keep credentials in environment/secrets management.

```

INSTALL httpfs;
LOAD httpfs;

SELECT COUNT(*)
  
```

```
FROM read_parquet(
  's3://activity-archive/activity_logs/year=2026/month=08/*.parquet'
);
```

For MinIO or another S3-compatible service, configure the endpoint and credentials according to that provider's current documentation. Keep buckets private.

9. Historical Query Examples

Total events

```
SELECT COUNT(*) AS total_events
FROM read_parquet('activity_logs/**/*.parquet');
```

Events by action

```
SELECT action, COUNT(*) AS total
FROM read_parquet('activity_logs/**/*.parquet')
GROUP BY action
ORDER BY total DESC;
```

Monthly activity

```
SELECT date_trunc('month', created_at) AS month,
       COUNT(*) AS total
FROM read_parquet('activity_logs/**/*.parquet')
GROUP BY month
ORDER BY month;
```

Tenant activity

```
SELECT tenant_id, COUNT(*) AS total
FROM read_parquet('activity_logs/**/*.parquet')
WHERE created_at >= '2025-01-01'
      AND created_at < '2026-01-01'
GROUP BY tenant_id
ORDER BY total DESC;
```

User history

```
SELECT *
FROM read_parquet('activity_logs/**/*.parquet')
WHERE tenant_id = 10
      AND user_id = 123
      AND created_at >= '2025-01-01'
      AND created_at < '2026-01-01'
ORDER BY created_at DESC
LIMIT 1000;
```

10. JSON Properties

Laravel activity logs often contain event-specific JSON properties. Keep frequently queried attributes as typed columns; use JSON for less-common event details.

```
{
  "event_id": "01J...",
  "tenant_id": 10,
  "user_id": 123,
  "action": "contact.updated",
  "subject_id": 456,
  "created_at": "2026-08-19T15:20:00Z",
  "properties": {
    "old": {"name": "John"},
    "new": {"name": "John Smith"}
  }
}
SELECT properties->'new'->'name' AS new_name
FROM read_parquet('activity_logs/**/*.parquet')
WHERE action = 'contact.updated';
```

11. Laravel Integration

Laravel should own authentication, authorization, tenant isolation, report orchestration and API responses. Keep DuckDB SQL in a dedicated analytics service/worker.

```
interface HistoricalActivityAnalytics
{
    public function activitySummary(
        int $tenantId,
        string $from,
```

```

        string $to
    ): array;
}
Next.js
  |
  v
Laravel API
  |
  v
Redis Queue
  |
  v
Python Analytics Worker
  |
  v
DuckDB
  |
  v
Parquet / Object Storage

```

Do not expose object-storage credentials or DuckDB internals to Next.js. Laravel remains the authorization boundary.

12. Historical Report API

```

POST /api/activity/reports

{
  "from": "2022-01-01",
  "to": "2026-08-19",
  "type": "monthly_summary"
}

```

Return immediately:

```

{
  "report_id": "rep_01...",
  "status": "queued"
}

```

Then poll:

```

GET /api/activity/reports/rep_01...
{
  "report_id": "rep_01...",
  "status": "completed",
  "download_url": "..."
}

```

For large exports, generate the file asynchronously and return a temporary signed object-storage URL.

13. DuckDB Performance

- Read only needed columns. Avoid `SELECT *` for reports.
- Filter by date. Organize archive paths by date.
- Use Parquet. Avoid raw JSON as the main analytical archive.
- Keep files reasonably large. Avoid tiny-file explosions.
- Batch operations. Avoid one query per event.
- Benchmark. Test with representative data.

Example

```

-- Better
SELECT action, created_at
FROM read_parquet('activity_logs/2026/*.parquet')
WHERE created_at >= '2026-01-01';

-- Avoid when only two columns are required
SELECT *
FROM read_parquet('activity_logs/2026/*.parquet')
WHERE created_at >= '2026-01-01';

```

A good file layout is part of query performance because it can reduce the amount of data DuckDB needs to read.

14. Concurrency — When DuckDB Is Not Enough

DuckDB is excellent for batch reports and ad-hoc analytics, but it is not intended to replace a highly concurrent analytical server receiving hundreds of dashboard queries simultaneously.

- Good: scheduled reports, audit investigations, large exports, analyst queries.
- Consider ClickHouse: many simultaneous dashboards, continuous low-latency analytics, high query concurrency.

Evolution path: MySQL + Parquet + DuckDB → add ClickHouse only when measured concurrency/latency requires it.

15. Long-Term Retention

Age	MySQL	Object Storage	DuckDB
0–12 months	Primary	Archive copy	Optional
1–5 years	Usually removed	Primary history	Query on demand
5–10 years	No	Primary history	Query on demand
10–20+ years	No	History + backup	Query on demand

Object storage retains the data; DuckDB does not need to run continuously for the archive to exist.

16. Archive Lifecycle and MySQL Cleanup

1. Event written to MySQL.
2. Event reaches archive queue/outbox.
3. Parquet created.
4. Parquet uploaded.
5. Manifest/checksum recorded.
6. Row counts and date bounds verified.
7. Archive marked complete.
8. After safety period, old MySQL partition may be removed.

Example manifest

```
{
  "object": "activity_logs/year=2025/month=01/part-0001.parquet",
  "rows": 1250345,
  "min_created_at": "2025-01-01T00:00:00Z",
  "max_created_at": "2025-01-31T23:59:59Z",
  "schema_version": 1,
  "sha256": "..."
```

Never delete a MySQL partition simply because a Parquet file exists. Verify completeness first.

17. Backup and Disaster Recovery

- Enable object-storage versioning where available.
- Keep an independent copy in another location for critical archives.
- Use immutable/WORM retention where required.
- Maintain manifests and checksums.
- Periodically download a representative archive and query it with DuckDB.
- Back up Laravel configuration, secrets securely, and the MySQL database separately.

Object storage stores the history; DuckDB proves that the history is queryable.

18. Security and Tenant Isolation

- Keep object-storage buckets private.
- Use separate writer and reader identities.
- Never expose storage credentials to Next.js.

- Enforce tenant_id authorization in Laravel.
- Do not accept arbitrary file paths from HTTP requests.
- Encrypt storage/network traffic as required.

DuckDB should not be your authorization layer. Laravel decides what the caller may see.

19. Failure Scenarios

Object storage unavailable

- Keep archive jobs pending; retry; do not delete MySQL source data.

DuckDB worker crashes

- Retry the report; historical source remains in object storage.

Corrupt Parquet

- Manifest/checksum detects it; recreate from source/backup.

Object storage deleted

- Versioning/replication/immutable backup provides recovery.

MySQL old data removed accidentally

- Rehydrate required history from the durable archive if needed.

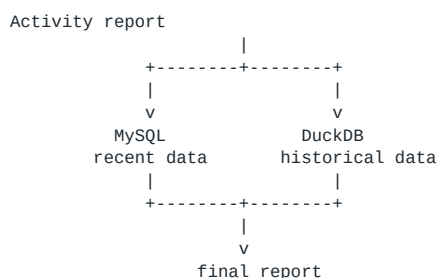
20. Capacity Planning

Raw growth example before compression/replicas/backups:

Monthly	5 years	10 years	20 years
10 GB	600 GB	1.2 TB	2.4 TB
50 GB	3 TB	6 TB	12 TB
100 GB	6 TB	12 TB	24 TB
250 GB	15 TB	30 TB	60 TB

Compression can reduce physical storage substantially, but measure your real activity data, especially the properties JSON column.

21. Combining Hot and Historical Data



For a date range crossing the hot/cold boundary, query MySQL for the recent portion and DuckDB for the older portion, then merge at the report layer. Avoid enormous raw unions; prefer aggregates or bounded pagination.

22. Exports

```

COPY (
  SELECT tenant_id, action, created_at
  FROM read_parquet('activity_logs/**/*.parquet')
  WHERE tenant_id = 10
     AND created_at >= '2025-01-01'
     AND created_at < '2026-01-01'
)

```

```

)
TO '/tmp/activity-2025.csv'
WITH (HEADER, DELIMITER ',');
COPY (
  SELECT *
  FROM read_parquet('activity_logs/**/*.parquet')
  WHERE tenant_id = 10
)
TO '/tmp/activity-tenant-10.parquet'
(FORMAT PARQUET, COMPRESSION ZSTD);

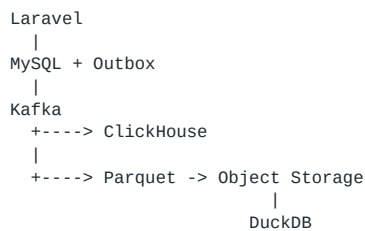
```

Upload completed exports to object storage and return a temporary signed URL through Laravel.

23. When to Add Kafka

Do not add Kafka solely because you use DuckDB. Redis Queue is enough for many moderate workloads.

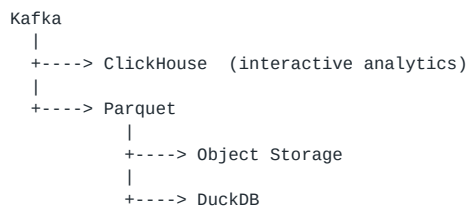
- Add Kafka for high sustained event throughput, multiple independent consumers, durable buffering, replay, and real-time pipelines.



24. When to Add ClickHouse

Add ClickHouse when historical analytics becomes a continuously available, high-concurrency workload rather than occasional reports.

- Many simultaneous dashboards.
- Near-real-time analytics.
- High query concurrency.
- Large aggregations running continuously.



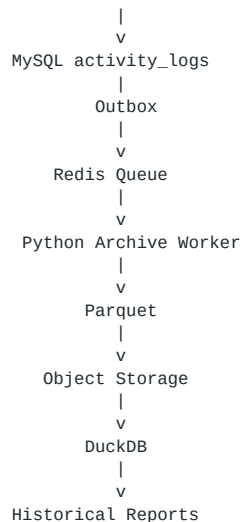
25. Production Checklist

- MySQL hot window is bounded.
- Archive jobs have retries and idempotency.
- Parquet schema is versioned.
- Archive objects are validated before source cleanup.
- Object storage is private and backed up.
- Historical report jobs are asynchronous.
- Tenant authorization is enforced in Laravel.
- Large reports use temporary signed downloads.
- DuckDB workers are supervised.
- Restore tests are performed periodically.
- Monitoring covers queue backlog, archive lag and report duration.

- Kafka/ClickHouse are added only when measured scale requires them.

26. Final Recommendation for Your Infrastructure

10.10.0.10 ■ HTTP
 10.10.0.11 ■ HTTP
 10.10.0.30 ■ HTTP + Redis
 10.10.0.40 ■ MySQL WRITE
 10.10.0.41 ■ MySQL READ



This is the recommended starting point because it avoids running Kafka and ClickHouse until you actually need them. It still gives you a path to 5–20+ years of retention and powerful historical SQL.

Evolution: MySQL + Outbox + Redis Queue + Parquet/Object Storage + DuckDB → Kafka when streaming/replay/multiple consumers require it → ClickHouse when high-concurrency analytics requires a dedicated server.

27. Reference Architecture Summary

Component	Role	Keep data?
MySQL	Transactional/hot activity	Recent window
Redis Queue	Async processing	Short-lived jobs
Parquet	Archive format	Yes
Object Storage	Durable archive	5–20+ years
DuckDB	Historical SQL analytics	No permanent archive needed
Kafka (optional)	Streaming/replay	Days/weeks typically
ClickHouse (optional)	Continuous analytical DB	Derived store

The key separation is: Object Storage = where long-term data lives; DuckDB = how you query it.

28. Official Documentation

DuckDB: <https://duckdb.org/docs/>

DuckDB Python client: <https://duckdb.org/docs/stable/clients/python/overview.html>

DuckDB Parquet: <https://duckdb.org/docs/stable/data/parquet/overview>

DuckDB HTTP/S3 access: https://duckdb.org/docs/stable/core_extensions/httpfs/overview

Apache Parquet: <https://parquet.apache.org/>

Laravel: <https://laravel.com/docs/>

Laravel queues: <https://laravel.com/docs/queues>

S3-compatible storage: use the official documentation for your chosen provider/MinIO version.

Use current official documentation for exact installation commands because release URLs and package instructions change over time.